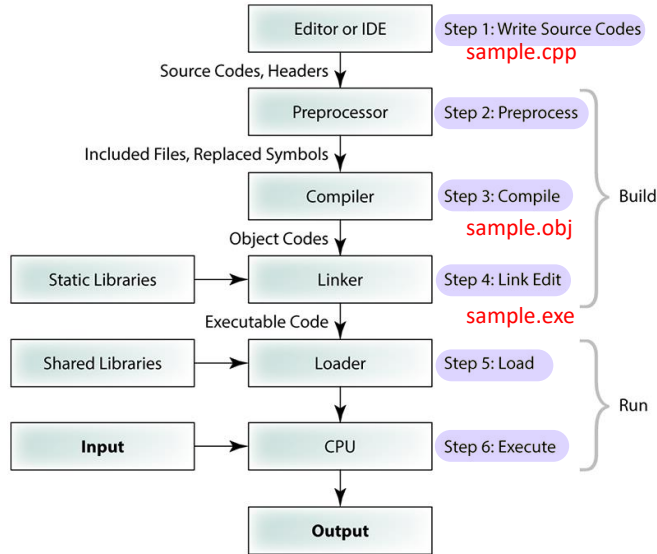




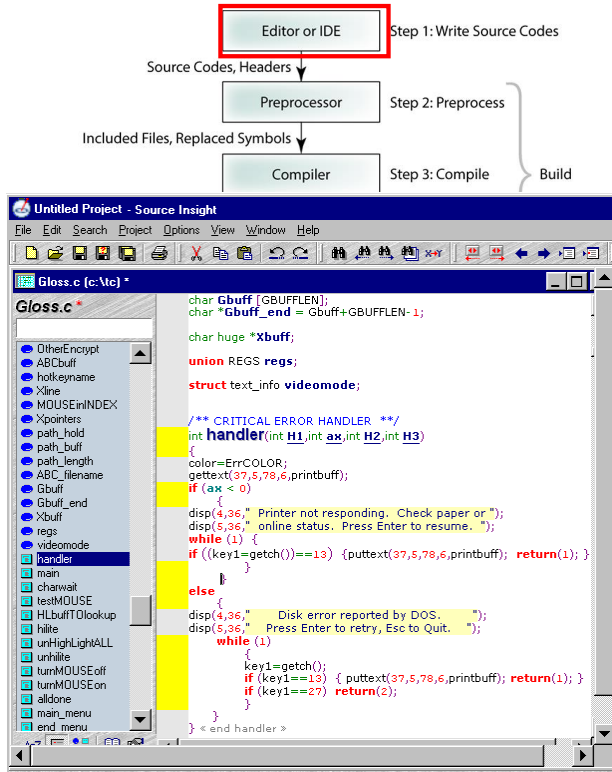
Programming Environment

Steps in developing a software



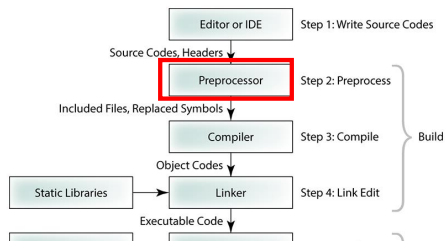
- the main idea of software development is to write an instruction for computer using a programming language
- it starts with programmer writing an instruction (code) for computer
- computer didn't understand the instruction which is in programming language
- it need to be converted to machine language by a compiler
- the files generated by the compiler will be combine with other related files to produce an executable file
- the executable file need to be loaded into a computer to make it run
- knowing the whole process will help in understanding and solving the problem during the development process

Source Code Editor



- Source code editors are a fundamental programming tool
 - as the fundamental job of programmers is to write and edit source code.
- A source code editor is a text editor program designed specifically for editing source code of computer programs
 - it make writing and reading the source code easier by differentiating the elements and routines so programmers can more easily look at their code.
- It may be a standalone application or it may be built into an integrated development environment (IDE) or web browser
- Source code editors have features specifically designed to simplify and speed up typing of source code, such as
 - syntax highlighting
 - indentation,
 - autocomplete
 - brace matching functionality.

Preprocessor



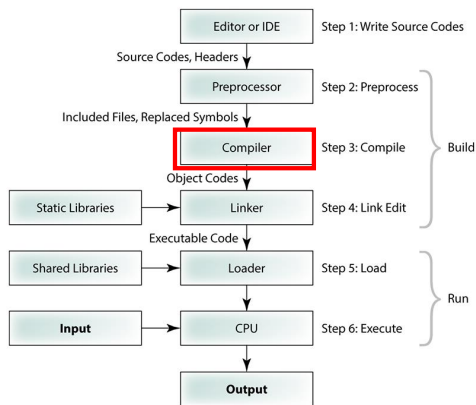
```
#include <iostream>
using namespace std;
#define PI 3.14159

int main()
{
    cout << "Value of PI:" << PI << endl;
    return 0;
}
```

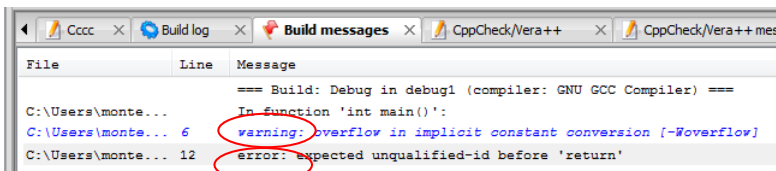
```
int main()
{
    cout << "Value of PI:" << 3.14159 << endl;
    return 0;
}
```

- in the preprocessor stage, the source code file is temporarily expanded to prepare for compilation
- preprocessor is a directive that starts with #
 - #define, #include, #if, #then, #else and #line are some of the preprocessors with which the compiler interacts
 - in C++ source code, programmer can include header files with extensions .h, .hxx, or .hpp, and sometimes with no extensions
- the first step of compilation, the compiler sends the code to a preprocessor
- When the preprocessor comes across #include, it searches for the specified header file to include in the compilation.
 - e.g. #include <iostream>
 - this preprocessor makes the library iostream available to the program by copying that file into the program
 - This is the library that includes, among other things, the definitions of cin and cout.
- The C++ preprocessor
 - copies the contents of the included header files into the source code file
 - generates macro code
 - and replaces symbolic constants defined using #define with their values.

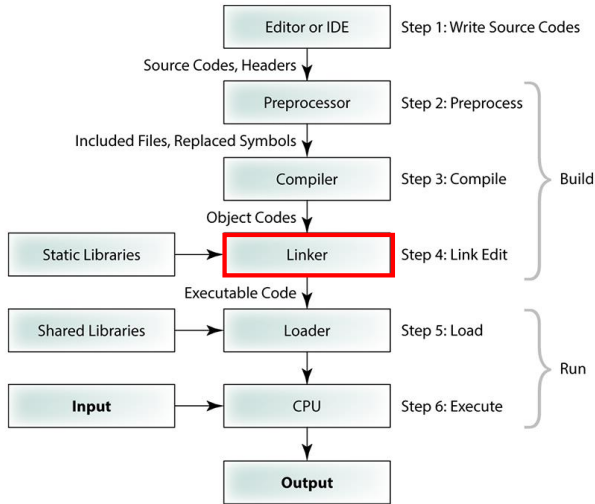
Compiler



- Compilation refers to the processing of source code files (.c, .cc, or .cpp) to produce an 'object' file
- compiler verify that the program obeys the rules of the programming language
- The compiler will catch certain kinds of mistakes and will write out an error message when it finds a mistake.
 - Compiler errors are usually syntactic in nature-called **syntax error**
 - a missing semicolon, an extra parenthesis.
 - sometimes the compiler will give **warning message**
 - not a violation of the programming language syntax rules, but that is unusual enough to indicate a likely mistake.
- if no error is found, it translates the program into the equivalent machine language
 - The equivalent machine language program is called an **object program** with the name .o or .obj
 - This step doesn't create anything the user can actually run



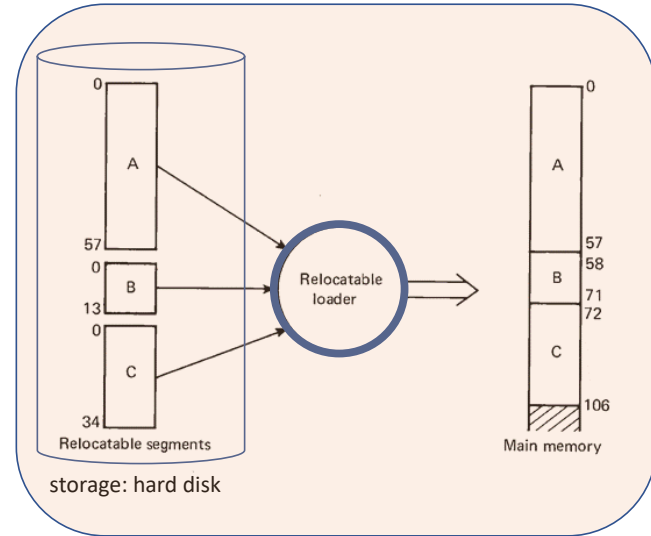
Linker



- **Linker** is a program that combines one or more object program with other programs in the library to create the executable code.
 - This prewritten code (program) resides in a place called the library.
 - It is the job of the linker to produce either a shared (dynamic) library or an executable file
- C++ is equipped with a library of prewritten code for performing common operations or sometimes-difficult tasks
 - e.g. library contains hardware-specific code for displaying messages on the screen and reading input from the keyboard
 - mathematical functions, such as calculating the square root of a number
 - **Static libraries** are archives of object code linked with an end-user application, that can become a part of an executable – use by linker
 - **Shared libraries** (dynamic libraries) are modules that contain data and functions that can be used by another application – use by loader
- The linker links all the object files by replacing all the references to undefined symbols with their correct addresses.
- The stage of linking may also produce some errors.
 - These errors are typically related to duplicate or missing definitions.
 - Duplicate definition errors occur when two libraries or object files contain the definition of the same symbol.
- Once the linker has finished, an *executable file* is created.

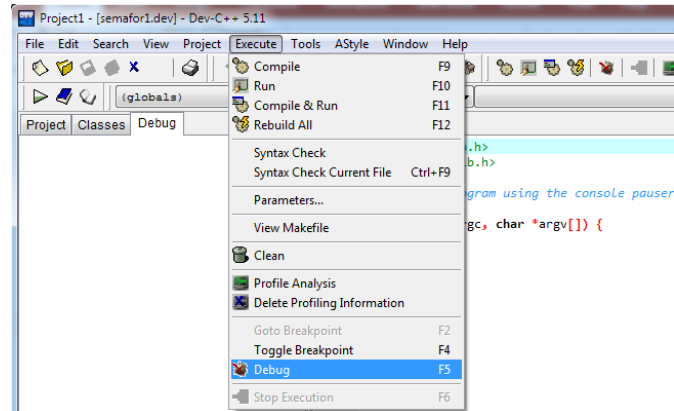
Loader

- Loader is a software (system utility) which load program from secondary memory to main memory and prepares them for execution
 - 2 types
 - Compile time loading – load static library – during linking
 - Run-time loading – load shared library dynamically
- during run-time loading, loader is used in order to obtain better memory space utilization
 - routine is not loaded until it is called
 - unused routine is never loaded
 - useful when large amount of code are needed to handle infrequently occurring case
 - e.g. error routine
- Every operating system has own loader and loader knows where to find dynamic library
- once loading is done operating system pass control to loaded program then execution take place.



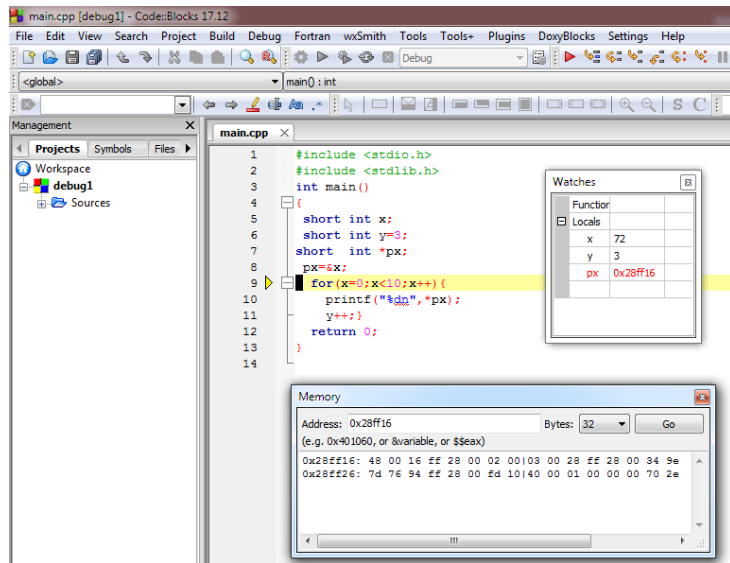
Integrated Development Environment (IDE)

- Development of a program requires a different type of software to edit, compile, link and run a program
- **integrated development environment (IDE)** is a software application that provides comprehensive facilities to computer programmers for software development
- IDE integrates all the software required to make the job of a programmer simpler.
- An IDE normally consists of a source code editor, build automation tools, and a debugger
- IDE integrate the following software
 - text editor to create the source program
 - compiler to compile a source code to object code
 - linker to link the object codes
 - program to execute the program.
 - debugger to debug the program
- Well-known IDEs used to create programs in the high-level language C++ include
 - Visual C++ , C++ Builder, Code::Block
 - in this class, Dev-C++ will be use

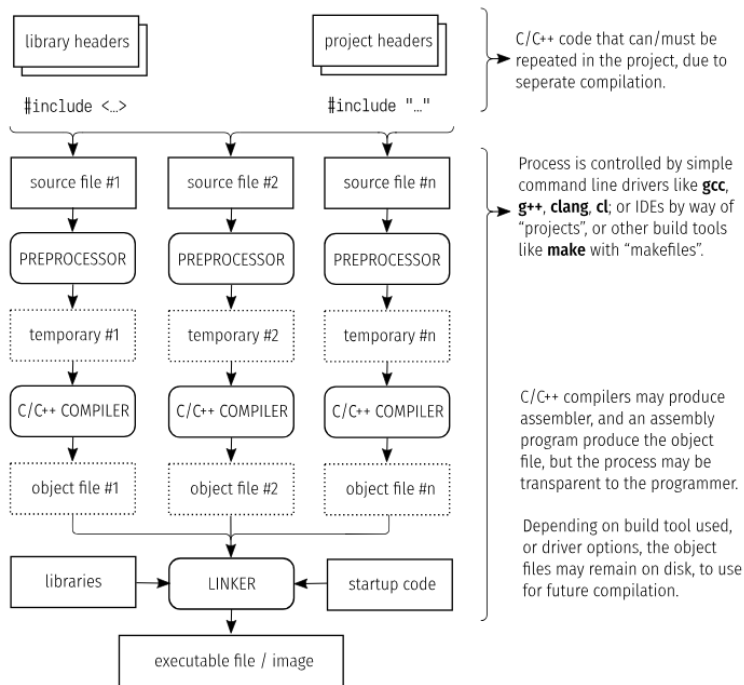


Debugging

- Debugging is the process of detecting and removing of existing and potential errors (also called as 'bugs') in a software code that can cause it to behave unexpectedly or crash
- Debugging tools (called debuggers) are used to identify coding errors at various development stages
 - used to reproduce the conditions in which error has occurred
 - then examine the program state at that time and locate the cause.
 - debugging component of an IDE typically provides the programmer with the capability to
 - single-stepping : trace the program execution step-by-step by evaluating the value of variables
 - breakpoint : stop the execution wherever required to get the value of variables
 - modify: change the value of variables or even change the contents of the line of code about to be executed
 - view: content of memory and variables
 - Debugger is use to catch a logical bugs – during execution
 - A logical bug is an error that even though the computer is able to carry out its instructions, it doesn't act as the programmer intended or the user expects.
- The final step of debugging is to test the correction or workaround and make sure it works.



Multi file compilation process



C/C++ MULTI-FILE COMPILATION PROCESS

- For a large program, full programs are not contained in a single file
 - full programs are separated into multiple files that contain different modules
 - each file need to be compiled separately
- why separate compilation ?
 - allows the creation of large programs without having to redo the compilation step every time a file is changed.
 - Instead, using so called "conditional compilation", it is necessary to compile only those source files that have changed; for the rest, the object files are sufficient input for the linker.
- simple to implement libraries of pre-compiled code
 - just create object files and link them just like any other object file